

Here are clean and well-structured notes from the transcript:

System Design Roadmap: Mid-Level to Senior Engineer

1. Overview & Core Skills

* **Transition from Mid-Level to Senior:** Shift focus from writing code to mastering high-level architectural design.

* **Senior Engineer Skills:**

* Designing systems/features from scratch.

* Making architectural decisions and tradeoffs.

* Working with rough requirements.

* Making systems performant.

* Optimizing data storage.

* Making decisions affecting customers and software.

* **Course Origin:** Developed by Hayek (Alex Simonyan), based on real-world experience and interviews.

2. Course Content Roadmap

* Foundations & Core Concepts of System Design.

* API Design.

* Databases.

* Caching, CDNs, & Load Balancing.

* Big Data Processing.

* Designing for Production.

* System Design Interviews.

3. Basic System Setup (Single Server)

* **Starting Small:** Begin with a simple setup for a single user, then gradually expand.

* **Single Server Architecture:**

* Web application, database, cache, and other components all run on one server.

* Visualizes core workings without added complexity.

* **Request Handling Flow:**

1. User (web browser/mobile app) attempts to access a domain (e.g., `app.demo.com`).

2. DNS (Domain Name System) maps the domain name to the server's IP address.

3. DNS sends the IP address back to the client.

4. Client sends an HTTP request to the server's IP address.

5. Server processes the request and sends back requested data (HTML for browser, JSON for mobile app).

* **Traffic Sources:**

* **Web Applications:** Server handles business logic, data storage, and presentation (HTML, CSS, JavaScript).

* **Mobile Applications:** Communication typically over HTTP, requesting data via API calls, often using JSON for responses.

* **Limitations:** Ideal for small user bases, but struggles under heavy traffic.

* **Key Takeaways:**

- * Start with a straightforward single server setup.
- * Understand request flow through the system.
- * Recognize unique demands for web vs. mobile applications.

4. Scaling - Separating Tiers

- * As user base grows, a single server is insufficient.
- * **Separation:** Divide into a Web Tier (handling web/mobile traffic) and a Data Tier (managing the database).
- * **Benefit:** Allows scaling each tier independently based on its specific load.

5. Database Selection

* **1. Relational Databases (RDBMS/SQL):**

- * **Structure:** Data organized in structured tables and rows.
- * **Examples:** PostgreSQL, MySQL, Oracle database, SQLite.
- * **Language:** Uses SQL (Structured Query Language).
- * **Components:**
 - * **Tables:** Fundamental building blocks, similar to spreadsheets.
 - * **Columns:** Fields or attributes of a table (e.g., ID, Name, Age, Email).
 - * **Rows:** Single records within a table.
- * **Advantages:**
 - * Supports complex join operations across multiple tables (e.g., connecting `Customers` and `Products` to form `Orders`).
 - * Provides robust data consistency and integrity, especially for transactions.
- * **ACID Properties of Transactions:**
 - * **Atomicity:** Entire transaction treated as a single unit; either completely succeeds or fails.
 - * **Consistency:** Transforms database from one valid state to another.
 - * **Isolation:** Concurrent transactions are isolated and don't interfere.
 - * **Durability:** Data persists even if the system or server fails.
- * **When to Use SQL:**
 - * Application data is well-structured with clear relationships (e.g., e-commerce: customers, orders).
 - * Needs strong consistency and transactional integrity (e.g., financial/banking systems).

* **2. Non-Relational Databases (NoSQL):**

- * **Suitability:** Flexible, fast access to large volumes of unstructured data.
- * **Examples:** Cassandra, MongoDB, Redis, Neo4j.
- * **Types:**
 - * **Document Stores (e.g., MongoDB):** Stores data in JSON-like documents, allowing complex data structures within a single record.
 - * **Wide Column Stores (e.g., Cassandra, Cosmos DB):** Data in tables, rows, and dynamic columns; handles massive scales, good for many write operations.
 - * **Graph Databases (e.g., Neo4j, Amazon Neptune):** Focus on storing entities and their relationships as graphs (e.g., product recommendations).

* **Key-Value Stores (e.g., Memcached, Redis):** Data stored in key-value pairs; simple, extremely fast (primarily RAM-based).

* **Advantages:**

* Handles highly dynamic and large data sets without rigid structure.

* Optimized for low latency and scalability.

* **When to Use NoSQL:**

* Application demands super low latency for quick responses.

* Data is unstructured or semi-structured (e.g., JSON objects), and relationships are less crucial.

* Requires flexible and scalable storage for massive data volumes (e.g., recommendation engine storing user activity).

6. Scaling Approaches

* **1. Vertical Scaling (Scale Up):**

* **Method:** Adding more resources (RAM, CPU) to an existing server.

* **Suitability:** Simple, works well for low or moderate traffic.

* **Limitations:**

* **Resource Limits:** Hard cap on how much can be added to a single server.

* **Lack of Redundancy:** Single server failure brings down the entire application.

* **2. Horizontal Scaling (Scale Out):**

* **Method:** Adding more servers (replicating existing servers) to share the load.

* **Suitability:** Generally more suitable for large-scale applications.

* **Advantages:**

* **Higher Fault Tolerance:** If one server fails, others continue serving users.

* **Better Scalability:** Easily add more servers as needed.

* **Implementation:** Requires a **Load Balancer** to distribute client requests.

7. Load Balancing

* **Purpose:** Distributes incoming traffic across multiple servers, ensuring no single server is overloaded.

* **Benefits:**

* Ensures even load distribution.

* Controls fault tolerance (reroutes traffic from failed servers).

* Makes applications more scalable (easily add/remove servers).

* **Load Balancing Strategies/Algorithms:**

* **Round Robin:** Distributes requests sequentially to each server in the pool; effective for servers with similar specifications.

* **Least Connections:** Directs traffic to the server with the fewest active connections; useful for variable session lengths.

* **Least Response Time:** Chooses the server with the lowest response time and fewest active connections; prioritizes fastest responses.

* **IP Hash:** Determines server based on a hash of the client's IP address; ensures consistent connection to the same server for a client (useful for stateful applications).

- * **Weighted Algorithms:** Servers are assigned weights based on capacity/performance; traffic distributed proportionally (e.g., weighted Round Robin, weighted Least Connections).
- * **Geographical Algorithms:** Directs requests to the server geographically closest to the user; reduces latency for global services.
- * **Consistent Hashing:** Uses a hash function to map users to servers on a "hash ring"; ensures the same client consistently connects to the same server, even with server additions/removals.
- * **Health Checks:**
 - * Load balancers continuously monitor servers by sending health check requests.
 - * Stop sending traffic to detected offline servers.
- * **Load Balancer Examples:**
 - * **Software Load Balancers:**
 - * Nginx: Popular, also acts as a web server, offers health checks.
 - * HAProxy: Open-source software.
 - * **Hardware Load Balancers:**
 - * F5 Load Balancer: High performance, feature-rich.
 - * Citrix.
 - * **Cloud-Based Load Balancers:**
 - * AWS Elastic Load Balancing (ELB).
 - * Azure Load Balancer.
 - * Google Cloud Load Balancing.
 - * Benefits: automatic security, auto-scaling, monitoring.

8. Single Point of Failure (SPOF)

- * **Definition:** Any component in a system whose failure will bring the entire system down.
- * **Examples:** A single database, a single load balancer.
- * **Problems with SPOFs:**
 - * **Reliability:** A single failure takes down the entire system, potentially causing business losses.
 - * **Scalability:** Systems with SPOFs struggle to scale as each component adds risk.
 - * **Security:** Attackers can target SPOFs to compromise the entire system.
- * **Avoiding Load Balancer SPOFs:**
 - * **Adding Redundancy:** Use more than one load balancer (e.g., active-standby or active-active setup).
 - * **Health Checks & Monitoring:** Continuously check the health of load balancers themselves.
 - * **Self-Healing Systems:** Automatically replace failed load balancer instances.

9. API Design Principles & Styles

- * **What is an API? (Application Programming Interface):**
 - * Defines how software components interact.
 - * Acts as a **contract** specifying requests, methods, endpoints, and expected responses.
- * **Abstraction Mechanism:** Hides implementation details while exposing functionality.
- * **Service Boundaries:** Defines clear interfaces between systems and components, allowing diverse systems to communicate.

* **Most Common API Styles:**

* **1. RESTful (Representational State Transfer):**

* **Approach:** Resource-based, uses standard HTTP methods (GET, POST, PUT/PATCH, DELETE).

* **Stateless:** Each request contains all necessary information to process it, no prior requests needed.

* **Use Cases:** Most commonly used in web and mobile applications.

* **Characteristics vs. GraphQL:**

* Resource-based endpoints (e.g., `/users``, `/products/{id}``).

* May require multiple requests for related data.

* Uses HTTP methods to define operations.

* Fixed response structures.

* Explicit versioning (e.g., `/api/v1/users``).

* Leverages HTTP caching.

* **2. GraphQL:**

* **Approach:** A query language allowing clients to request *exactly what they need*.

* **Endpoint:** Single endpoint for all operations.

* **Operations:**

* `Query``: Retrieving data (like GET in REST).

* `Mutation``: Updating data (like POST, PUT, PATCH, DELETE in REST).

* `Subscription``: Real-time communication.

* **Advantages:**

* Minimal round trips (single request can fetch all needed data).

* Avoids over-fetching or under-fetching.

* **Use Cases:** Complex UIs where different pages require varied and nested data.

* **Characteristics vs. REST:**

* Single endpoint (e.g., `/graphql``).

* Single request to get precise, aggregated data.

* Uses GraphQL query language for defining requests.

* Client specifies the desired response structure.

* Schema evolution typically without versioning (or field versioning).

* Leverages application-level caching.

* **3. gRPC (Google Remote Procedure Call):**

* **Approach:** High-performance RPC (Remote Procedure Call) framework.

* **Communication:** Uses Protocol Buffers for efficient data serialization.

* **Methods:** Defined as RPCs in `.proto`` files.

* **Features:** Supports streaming and bidirectional communication.

* **Use Cases:** Excellent for microservices and internal system communication (more efficient between servers than REST/GraphQL).

* **Four Essential API Design Principles:**

* **Consistent:** Use consistent naming, casing, and patterns across all endpoints.

* **Simple:** Focus on core use cases and intuitive design, making it usable without extensive documentation.

* **Secure:** Implement authentication, authorization, input validation, and rate limiting.

* **Performant:** Design for efficiency using caching strategies, pagination (limit, offset, cursor), minimized payloads, and reduced round trips.

* **Application Protocols Influence API Design:**

* **HTTP:** Enables RESTful capabilities, provides status codes for CRUD operations.

* **WebSockets:** Enable real-time, bidirectional communication (e.g., chat, video streaming).

* **HTTP/2 (used by gRPC):** Faster transport for efficient inter-service communication.

* **Choice Factors:** Limitations, strengths, performance, capabilities required by the API.

* **API Design Process:**

* **1. Understanding Requirements:**

* Identify core use cases and user stories.

* Define scope and boundaries (what's in/out of scope).

* Determine performance requirements and potential bottlenecks.

* Establish security constraints (authentication, authorization, rate limiting).

* **2. Design Approaches:**

* **Top-Down:** Start with high-level requirements and workflows, then define endpoints (common in interviews).

* **Bottom-Up:** Design API based on existing data models and capabilities (common in company settings).

* **Contract-First:** Define API contract (requests/responses) before implementation (similar to top-down).

* **3. Lifecycle Management:**

* **Design:** Discuss requirements, define outcomes.

* **Development & Testing:** Implement and test locally.

* **Deployment & Monitoring:** Deploy to staging/production, monitor performance.

* **Maintenance:** Ensure long-term usability and updateability.

* **Deprecation & Retirement:** Phase out old API versions or entire APIs.

10. API Protocols (Deep Dive)

* **Role in Network Stack:** Application layer protocols sit at the top, defining message formats, request-response patterns, connection management, and error handling, built on top of transport layer protocols (TCP/UDP).

* **1. HTTP (Hypertext Transfer Protocol):**

* **Foundation of Web APIs:**

* **Client-Server Interaction:** Client sends request (method, URL, HTTP version, host, authentication), server sends response (HTTP version, status code, content type, headers).

* **HTTP Methods:**

* `GET`: Retrieve data (safe, idempotent).

- * `POST`: Create data (changes state, not idempotent).
- * `PUT`/`PATCH`: Update data (PUT replaces fully, PATCH partially updates).
- * `DELETE`: Remove data.
- * **Status Codes:**
- * `2xx` (Success): 200 OK, 201 Created, 204 No Content.
- * `3xx` (Redirection).
- * `4xx` (Client Errors): 400 Bad Request, 401 Unauthorized, 404 Not Found.
- * `5xx` (Server Errors): 500 Internal Server Error.
- * **Common Headers:** Content-Type, Authorization, Accept, Cache-Control, User-Agent.
- * **2. HTTPS (HTTP Secure):**
- * **Enhancement:** HTTP with TLS/SSL encryption.
- * **Security:** Adds a security layer, protects data in transit.
- * **Benefits:** Encrypted data, data integrity, user authentication, SEO benefits.
- * **Standard:** Always use HTTPS in production.
- * **3. WebSockets:**
- * **Purpose:** Solves limitations of HTTP (high latency, wasted bandwidth from polling) for real-time communication.
- * **Mechanism:** Initial HTTP handshake establishes a persistent, bidirectional connection.
- * **Communication:** Server can independently push data to the client; client can still send requests.
- * **Benefits:** Real-time data with minimal latency, reduced bandwidth usage.
- * **Use Cases:** Chat applications, video streaming, live updates.
- * **4. AMQP (Advanced Message Queuing Protocol):**
- * **Purpose:** Enterprise messaging protocol for message queuing and guaranteed delivery.
- * **Model:** Producer publishes messages to a message broker (queues); consumer pulls messages when ready.
- * **Benefits:** Asynchronous communication, decoupling of services, consumer capacity management.
- * **Exchange Types:** Direct, fan out, topic-based.
- * **5. gRPC (Google Remote Procedure Call):**
- * **Purpose:** High-performance RPC framework.
- * **Transport:** Uses HTTP/2 (requires client support).
- * **Serialization:** Uses Protocol Buffers.
- * **Features:** Built-in streaming capacities.
- * **Use Cases:** Most commonly used for inter-server communication, especially in microservices architectures.
- * **Choosing the Right Protocol:**
- * **Interaction Patterns:** Request-response (HTTP), real-time (WebSockets), asynchronous (AMQP).
- * **Performance Requirements:** High-performance inter-service (gRPC).

* **Client Compatibility:** Browser support for HTTP/2 can limit gRPC for client-server.

* **Payload Size & Encoding.**

* **Security Needs.**

* **Developer Experience:** Tooling and documentation availability.

11. Transport Layer Protocols (TCP/UDP)

* **Role:** Sit below application layer protocols, handle data movement between machines over the network.

* **1. TCP (Transmission Control Protocol):**

* **Characteristics:** Reliable, but slower.

* **Delivery Guarantee:** Ensures all data packets arrive, resends lost packets, reorders out-of-order packets.

* **Connection-Based:** Establishes a connection via a **three-way handshake** before sending data.

1. Client sends SYN (synchronize sequence number).

2. Server sends SYN-ACK (synchronize-acknowledge).

3. Client sends ACK (acknowledge).

* **Overhead:** Adds overhead for accuracy and reliability.

* **Use Cases:** Applications requiring high data accuracy and reliability (e.g., banking, emails, payments, HTTP/HTTPS).

* **2. UDP (User Datagram Protocol):**

* **Characteristics:** Fast, efficient, but unreliable.

* **Delivery:** No delivery guarantee; packets can be lost, no reordering.

* **Connectionless:** No handshake or connection establishment.

* **Overhead:** Less overhead due to lack of guarantees.

* **Use Cases:** Applications where speed is critical and some data loss is acceptable (e.g., video calls, online games, live streams).

* **Choosing Between TCP and UDP:**

* **Reliability & Safety:** Choose TCP.

* **Speed & Lightweight (with acceptable data loss):** Choose UDP.

12. RESTful API Design Best Practices

* **1. Resource Modeling and URL Design:**

* **Resources as Nouns:** Model business domains as nouns, not verbs (e.g., `products`, `orders`).

* **Collections vs. Individual Items:**

* `/api/products``: Returns a collection of products.

* `/api/products/{id}``: Returns an individual product.

* **Plural Nouns:** Always use plural nouns for collection resources (e.g., `users`, not `user`).

* **Clearly Identifiable URLs:** Resources should be clear and predictable.

* **Nested Resources:** Clearly define relationships (e.g., `/products/{id}/reviews``).

* **2. Filtering, Sorting, and Pagination:**

* **Implementation:** Use query parameters.

- * **Filtering:** `GET /api/products?category=electronics&in_stock=true``.
- * **Sorting:** `GET /api/products?sort=price_asc``.
- * **Pagination:**
 - * `GET /api/products?page=2&limit=10``
 - * `GET /api/products?offset=20&limit=10``
 - * Cursor-based pagination (e.g., `?cursor=hashed_page_id``).
- * **Benefits:** Saves bandwidth, improves performance (server & frontend), offers frontend flexibility.
- * **3. HTTP Methods for CRUD Operations:**
 - * `GET /api/{resource}`` or `/api/{resource}/{id}``: Read data (safe, idempotent).
 - * `POST /api/{resource}``: Create a resource (changes state, not idempotent).
 - * `PUT /api/{resource}/{id}``: Replace an entire resource (updates, idempotent).
 - * `PATCH /api/{resource}/{id}``: Partially update a resource (updates).
 - * `DELETE /api/{resource}/{id}``: Remove a resource (changes state, idempotent).
- * **4. Status Codes and Error Handling:**
 - * Use appropriate HTTP status codes for responses.
 - * **2xx (Success):** 200 OK (general success), 201 Created (resource created), 204 No Content.
 - * **3xx (Redirection):**
 - * **4xx (Client Errors):** 400 Bad Request (invalid request), 401 Unauthorized (not authenticated), 404 Not Found (resource not found).
 - * **5xx (Server Errors):** 500 Internal Server Error (unexpected server-side error).
- * **5. Versioning:**
 - * Include API version in the URL (e.g., `/api/v1/products``).
 - * **Benefit:** Allows backward compatibility when new API versions are deployed, preventing breakage for existing clients.

13. GraphQL API Design

- * **Why GraphQL Exists:**
 - * To solve issues of over-fetching (receiving too much data) or under-fetching (receiving too little, requiring multiple requests) common in REST.
 - * Client specifies the exact data shape it needs from a single endpoint.
- * **Schema Design and Type System:**
 - * **Schema:** Contract between client and server, defining available data and operations.
 - * **Types:** Define data structures (e.g., `User { id: ID, name: String, posts: [Post] }`).
 - * **Queries:** Define how to read data (e.g., `user(id: ID): User``).
 - * **Mutations:** Define how to modify data (e.g., `createUser(name: String): User``).
 - * **Subscriptions:** Define real-time data streams.
 - * **Best Practice:** Schema should mirror the domain model, be intuitive and flexible.
- * **Querying and Mutating Data:**
 - * **Queries:** Clients specify precisely which fields they need, including nested relationships (e.g., `query { user(id: "123") { name, posts { title } } }`).

* **Mutations:** Clients send data to modify resources and specify what data they want back (e.g., `{ createPost(title: "New Post", body: "...") { id, title } }`).

* **Error Handling:**

* **HTTP Status:** GraphQL always returns a `200 OK` HTTP status, even if there's an error.

* **Error Field:** Errors are indicated within an `errors` field in the JSON response body.

* **Partial Data:** Partial data can be returned alongside errors.

* **Structure:** Error objects include status codes, messages, and paths (e.g., `{ errors: [{ status: 404, message: "Not Found", path: "user" } ] }`).

* **GraphQL Design Best Practices:**

* Keep schemas small and modular.

* Avoid deeply nested queries (implement query depth limits).

* Use meaningful naming for types and fields.

* Use input types for mutations.

14. Authentication

* **Definition:** Verifies the identity of the requester (user or service) attempting to access a system. It answers "Who is the user?"

* **Common Confusions:**

* JWT is a token *format*, not an authentication method.

* Bearer authentication is a *pattern*, not a specific method.

* OAuth2 is an *authorization framework*, not an authentication method.

* Single Sign-On (SSO) is a *user experience pattern*, not an authentication method.

* **Authentication Methods:**

* **1. Basic Authentication:**

* **Mechanism:** Client sends `Authorization` header with Base64-encoded `username:password`.

* **Security:** Insecure without HTTPS. Rarely used in production today.

* **2. Digest Authentication:**

* **Mechanism:** Similar to Basic, but uses MD5 hashing of password.

* **Security:** Slightly better than Basic, but still outdated and rarely used.

* **3. API Key Authentication:**

* **Mechanism:** Unique key generated per client; sent with `Authorization` or `X-API-Key` header. Server looks up key in DB, verifies validity, and checks scopes.

* **Limitations:** If key leaks, anyone can use it. No built-in expiration. API keys are random strings, no embedded information.

* **4. Session-Based Authentication (Traditional Web):**

* **Mechanism:** User logs in, server creates a session in session storage (e.g., Redis). Server sends a session ID, client stores it in an HTTP-only cookie. Subsequent requests include the cookie, server looks up session to verify.

* **Challenges:** Stateful (server must remember sessions), harder to scale for APIs and distributed systems.

* **5. Token-Based Authentication (Modern):**

* **Mechanism:** Client sends a token with each request (e.g., `Authorization: Bearer <token>`).

- * **Bearer Token:** "Whoever has this token gets access."
- * **JWT (JSON Web Token):** A common type of bearer token. A signed JSON object containing claims (user ID, email, expiration, roles, permissions).
- * **Stateless:** Server validates the JWT signature locally without needing a database lookup (highly scalable). Short-lived JWTs reduce database load.
- * **Access Tokens & Refresh Tokens:**
- * **Access Tokens:** Short-lived (e.g., 15 mins - 1 hour); used for API calls.
- * **Refresh Tokens:** Long-lived (days/weeks); used to obtain new access tokens when the current one expires.
- * **Flow:** User logs in, receives both. Access token for API, refresh token stored in HTTP-only cookie. When access token expires, refresh token used to get a new one from the auth server.
- * **6. OAuth 2 (Authorization Framework):**
- * **Focus:** **Authorization**, not authentication. Answers "what can this app access on behalf of the user?"
- * **Mechanism:** Delegated authorization. User grants an application permission to access their resources on another service (e.g., app accessing Google Drive).
- * **Flow:** User allows app -> redirects to consent screen -> app gets authorization code -> app exchanges code for an **access token** from the service's OAuth endpoint. This access token proves the app's permission, but does not identify the user.
- * **7. OpenID Connect (Authentication on top of OAuth 2):**
- * **Focus:** Adds **authentication** on top of OAuth 2. Answers "who is the user?"
- * **Mechanism:** User signs in via an app (e.g., "Sign in with Google").
- * **Flow:** User signs in -> authorization provider returns authorization code -> app exchanges code for an **access token** (for OAuth 2 authorization) and an **ID token**.
- * **ID Token:** A JWT containing the user's identity (email, username, user ID). App verifies ID token, confirming user's identity.
- * **Benefits:** Modern, secure, scalable solution used for "Sign in with Google/GitHub/Microsoft" options.
- * **8. Single Sign-On (SSO - User Experience Pattern):**
- * **Definition:** Log in once, access multiple services (e.g., logging into Google grants access to Gmail, Drive, YouTube).
- * **Mechanism:** Uses identity protocols (SAML, OpenID Connect) to validate sessions. A global session is stored, and an SSO cookie allows access to other services without re-authentication.
- * **Identity Protocols Used by SSO:**
- * **SAML (Security Assertion Markup Language):** XML-based protocol, common in enterprise and legacy systems.
- * **OpenID Connect:** Modern, JSON Web Token-based, used by major providers like Google.

15. Authorization

- * **Definition:** Happens **after** authentication. Determines **what** the authenticated user can do in the system, which resources or actions they have permissions for. Controls security and privacy.
- * **Common Authorization Models:**
- * **1. Role-Based Access Control (RBAC):**

* **Mechanism:** Users are assigned to roles (e.g., admin, editor, viewer). Each role has a predefined set of permissions (e.g., `admin` has full access, `viewer` only read access).

* **Advantages:** Most common and widely used (e.g., GitHub, Stripe dashboards, CMS tools). Simple to manage.

* **2. Attribute-Based Access Control (ABAC):**

* **Mechanism:** Access decisions based on user attributes (department, age), resource attributes (confidentiality, owner), and environment conditions (time of day, location, device type). Policies define conditions for access.

* **Advantages:** More flexible and granular than RBAC.

* **Challenges:** More complex, requires good policy management, potential for conflicts.

* **3. Access Control Lists (ACL):**

* **Mechanism:** Each resource has its own explicit list of permissions, specifying which users can access it and what actions they can perform (e.g., User Alice has read-only access to Document A).

* **Advantages:** High control over individual resources.

* **Challenges:** Highly specific and user-centric, harder to scale for systems with millions of users/objects unless managed carefully (e.g., Google Drive for documents).

* **Enforcing Authorization (Technologies):**

* **OAuth 2 (Delegated Authorization):**

* **Mechanism:** Used when one service needs to access another service's resources on behalf of a user (e.g., Vercel accessing GitHub repositories). User grants specific permissions, and an access token representing those permissions is issued. The third-party app uses this token, not the user's credentials.

* **JWT / Bearer Tokens & Permission Logic:**

* **Mechanism:** After authentication, a JWT (bearer token) is issued. This token carries user identity and claims (user ID, roles, scopes, expiration). The backend server validates the token and applies permission logic based on the roles/scopes embedded in the token.

* **Key Distinction:** Tokens are *mechanisms* for carrying authorization information; RBAC, ABAC, ACL are the *models* that define what is allowed.

* **Summary:** Authorization controls "what they can do" in the system. Models have tradeoffs and are often combined for flexibility and security.

16. API Security

* **Seven Proven Techniques to Protect APIs:**

* **1. Rate Limiting:**

* **Mechanism:** Controls the number of requests a client can make within a given time period.

* **Prevention:** Prevents system overwhelm, brute-force attacks, and DDoS attacks.

* **Implementation:** Can be set per endpoint, per user/IP address, or globally.

* **2. CORS (Cross-Origin Resource Sharing):**

* **Mechanism:** Browser security feature controlling which domains can make requests to your API.

* **Prevention:** Prevents malicious websites from tricking users' browsers into making unauthorized requests.

* **Implementation:** Configure your API to only allow requests from trusted frontend domains.

* **3. SQL & NoSQL Injections:**

* **Mechanism:** Attackers include malicious input directly into database queries to read, modify, or delete data.

* **Prevention:** Use parameterized queries or ORM (Object-Relational Mapping) safeguards to separate code from user input.

* **4. Firewalls:**

* **Mechanism:** Acts as a gatekeeper, filtering malicious traffic from legitimate traffic.

* **Prevention:** Blocks requests with unknown attack patterns (e.g., suspicious SQL keywords, unusual HTTP methods).

* **Example:** AWS Web Application Firewall (WAF).

* **5. VPNs (Virtual Private Networks):**

* **Mechanism:** Creates a secure, private network. APIs within this network are only accessible by users also connected to the same VPN.

* **Use Cases:** Protecting internal tools, admin dashboards, or sensitive APIs that should not be public-facing.

* **6. CSRF (Cross-Site Request Forgery):**

* **Mechanism:** Tricks a logged-in user's browser into making unwanted requests to an API (e.g., a hidden form submitting a bank transfer using the user's session cookie).

* **Prevention:** Use CSRF tokens in combination with session cookies. The server validates if the token in the request matches the one it expects.

* **7. XSS (Cross-Site Scripting):**

* **Mechanism:** Attackers inject malicious scripts into web pages (e.g., via comment sections). When other users view the page, their browser executes the injected script, potentially stealing cookies or modifying content.

* **Prevention:** Input validation and output encoding to neutralize malicious scripts before they are stored or rendered.